

lab4_N-Grams.py

Spiegazione dettagliata, riga per riga, del codice del Laboratorio 4

"Modeling social media and literary language with N-grams"

Corso di Tecnologie del Linguaggio Naturale — Prof. Daniele Radicioni, UniTO

File analizzato: TLN/Parte2Radicioni/lab4_N-Grams.py (479 righe)

Obiettivo del laboratorio: confrontare, tramite modelli n-gram, il linguaggio dei social media (Twitter) con quello letterario (il romanzo *Emma* di Jane Austen), usando stima MLE e smoothing Laplace, generazione di testo, perplexity e classificazione per dominio.

Come leggere questo documento: ogni sezione riprende un blocco del codice (nell'ordine in cui appare nel file) e lo espone in una tabella a tre colonne: numero di riga, codice sorgente, spiegazione. Dopo ogni tabella è riportato un riquadro **"Esempio di output reale"** con i valori effettivamente ottenuti eseguendo lo script, per collegare ogni blocco di codice a ciò che produce concretamente.

0. Intestazione, import e setup (righe 1–29)

Questo blocco prepara l'ambiente: importa tutte le librerie necessarie, configura l'encoding dell'output e scarica le risorse NLTK (corpora e tokenizer) che verranno usate nel resto dello script.

Riga	Codice	Spiegazione
4	<code>import nltk</code>	Libreria principale per l'NLP: fornisce i corpora (Twitter, Gutenberg) e i modelli linguistici n-gram usati più avanti.
10	<code>from nltk.lm import MLE, Laplace</code>	Le due classi di modello n-gram confrontate nel laboratorio: MLE (stima pura) e Laplace (con smoothing add-one).
13–15	<code>matplotlib.use('Agg')</code>	Backend non interattivo, necessario per salvare grafici su file senza display grafico disponibile.
18	<code>sys.stdout.reconfigure(encoding='utf-8')</code>	Evita <code>UnicodeEncodeError</code> su Windows quando si stampano simboli come → o ✓.
21–25	<code>nltk.download(...)</code>	Scarica corpora e tokenizer necessari, idempotente: rende lo script eseguibile "a freddo" ovunque.

► Esempio di output reale

```
[nltk_data] Downloading package twitter_samples to ...
[nltk_data]   Package twitter_samples is already up-to-date!
[nltk_data] Downloading package gutenberg to ...
[nltk_data]   Package gutenberg is already up-to-date!
Import completati!
```

Al primo avvio (nessuna cache locale) ogni riga di download mostrerebbe l'avanzamento del download invece di "already up-to-date"; nelle esecuzioni successive è quasi istantaneo.

1. Caricamento dei due corpora (righe 31–40)

Vengono caricati i due dataset che rappresentano i due "domini linguistici" da confrontare: un insieme di tweet e il romanzo *Emma* di Jane Austen.

Riga	Codice	Spiegazione
32–34	<code>tweets_pos =</code> <code>twitter_samples.strings('positive_tweets.json')</code> <code>tweets_neg =</code> <code>twitter_samples.strings('negative_tweets.json')</code> <code>tweets_raw = tweets_pos + tweets_neg</code>	10.000 tweet totali (5000 positivi + 5000 negativi): il sentiment non è rilevante qui, serve solo un campione ampio di linguaggio "da social".
36	<code>testo_letterario_raw = gutenberg.raw('austen-emma.txt')</code>	L'intero romanzo come stringa unica; verrà spezzato in frasi più avanti.
38–40	<code>print(f"Tweet caricati: {len(tweets_raw)}")</code> <code>print(f"Esempio positivo: {tweets_pos[0]}")</code> <code>print(f"Esempio negativo: {tweets_neg[0]}")</code>	Sanity check: verifica visiva che il caricamento sia andato a buon fine.

► Esempio di output reale

```
Tweet caricati: 10000
Esempio positivo: #FollowFriday @France_Inte @PKuchly57 @Milipol_Paris for being top engaged
members in my community this week :)
Esempio negativo: hopeless for tmr :(
```

2. Pulizia del testo (righe 42–62)

Due funzioni di pulizia, una per dominio, per rimuovere elementi (URL, menzioni, punteggiatura, maiuscole) che gonfierebbero artificialmente il vocabolario.

Riga	Codice	Spiegazione
44	<code>testo = re.sub(r'http\S+', '', testo)</code>	Rimuove gli URL.
45	<code>testo = re.sub(r'@\w+', '', testo)</code>	Rimuove le menzioni (@utente) per intero.
46	<code>testo = re.sub(r'#', '', testo)</code>	Rimuove solo il simbolo #, mantenendo la parola dell'hashtag (spesso semanticamente utile).
47	<code>testo = re.sub(r'[^\w\s]', '', testo)</code>	Elimina tutto ciò che non è lettera o spazio: numeri, punteggiatura, emoji.
48	<code>testo = testo.lower().strip()</code>	Minuscolizza e rimuove spazi residui.
57	<code>tweets_puliti = [t for t in tweets_puliti if len(t) > 0]</code>	Scarta i tweet svuotati dalla pulizia (es. solo URL/menzioni).

► Esempio di output reale

=== PULIZIA ===

Tweet PRIMA: #FollowFriday @France_Inte @PKuchly57 @Milipol_Paris for being top engaged members in my community this week :)

Tweet DOPO: followfriday for being top engaged members in my community this week

Menzioni e hashtag-marker sono spariti, "followfriday" è rimasto attaccato (era preceduto da # senza spazio), la faccina :) è stata eliminata dal filtro sui soli caratteri alfabetici.

3. Tokenizzazione (righe 64–81)

Due rappresentazioni parallele: un flusso unico di token per le statistiche aggregate, e sequenze separate per unità testuale (tweet singolo/frase singola), indispensabili per non far attraversare i bigrammi tra un tweet/frase e il successivo.

Riga	Codice	Spiegazione
65–66	<code>tokens_tweet = word_tokenize(testo_tweet_unito)</code> <code>tokens_lett = word_tokenize(testo_lett_pulito)</code>	Liste piatte di token per le statistiche generali.
72	<code>sequenze_tweet = [word_tokenize(t) for t in tweets_puliti]</code>	Tokenizza ogni tweet singolarmente: struttura usata per l'addestramento dei modelli.
73–74	<code>frasi_lett_raw = sent_tokenize(testo_letterario_raw)</code> <code>sequenze_lett = [word_tokenize(pulisci_letterario(f)) for f in frasi_lett_raw]</code>	Segmenta il romanzo in frasi sul testo <i>grezzo</i> (la punteggiatura serve a <code>sent_tokenize</code>), poi pulisce e tokenizza ogni frase.

► Esempio di output reale

=== TOKENIZZAZIONE ===

Token Twitter: 94706

Token Letterario: 158267

Parole uniche Twitter: 12024

Parole uniche Letterario: 9305

Il romanzo ha più token totali ma meno parole uniche: vocabolario più ripetitivo (registro narrativo), mentre Twitter è lessicalmente più vario nonostante il volume minore di token.

4. Stopword e bigrammi "di contenuto" (righe 83–98)

Bigrammi più frequenti dopo la rimozione delle stopwords, per far emergere i temi ricorrenti invece delle sole strutture grammaticali.

Riga	Codice	Spiegazione
83	<code>stop = set(stopwords.words('english'))</code>	Set per test di appartenenza O(1), controllato per ogni token.
89–90	<code>bi_tweet_filtrati = Counter(ngrams(tokens_tweet_filtrati, 2))</code> <code>bi_lett_filtrati = Counter(ngrams(tokens_lett_filtrati, 2))</code>	Conta i bigrammi formati dopo il filtro stopwords: può accoppiare parole non originariamente adiacenti (limite metodologico noto).

► Esempio di output reale

=== BIGRAMMI SENZA STOPWORDS ===

```
TWITTER (top 10):
('wan', 'na')           -> 123 volte
('follow', 'please')    -> 81 volte
('gon', 'na')           -> 72 volte
('follow', 'amp')       -> 64 volte
('amp', 'follow')       -> 63 volte
('follow', 'u')         -> 62 volte
('u', 'back')           -> 62 volte
('arrived', 'new')      -> 60 volte
('unfollowers', 'via')  -> 60 volte
('happy', 'birthday')   -> 54 volte

LETTERARIO (top 10):
('mr', 'knightley')     -> 248 volte
('mrs', 'weston')       -> 222 volte
('mr', 'elton')         -> 184 volte
('miss', 'woodhouse')   -> 148 volte
('mr', 'weston')        -> 138 volte
('frank', 'churchill')  -> 123 volte
('mrs', 'elton')        -> 120 volte
('every', 'thing')      -> 119 volte
('miss', 'fairfax')     -> 105 volte
('mr', 'woodhouse')     -> 104 volte
```

Twitter: gergo e inviti a seguire ("follow please", "wanna"). Letterario: quasi solo nomi propri dei personaggi del romanzo — coerente con lo stile narrativo di Austen.

5. Bigrammi e trigrammi "con" stopwords (righe 100–120)

Stessa estrazione sul testo integrale (stopword include), per confrontare lo stile sintattico piuttosto che i temi, sia a livello di bigrammi sia di trigrammi.

Riga	Codice	Spiegazione
101–104	<pre>bi_tweet = Counter(ngrams(tokens_tweet, 2)) tri_tweet = Counter(ngrams(tokens_tweet, 3))</pre>	Bigrammi/trigrammi su token non filtrati, per osservare le strutture grammaticali ricorrenti.

► Esempio di output reale

```
=== BIGRAMMI CON STOPWORDS (top 10) ===
TWITTER:
('thank', 'you')    -> 260 volte
('for', 'the')      -> 249 volte
('have', 'a')       -> 199 volte
('i', 'love')       -> 173 volte
('i', 'have')       -> 163 volte
('i', 'miss')       -> 160 volte
('thanks', 'for')   -> 158 volte
('love', 'you')     -> 155 volte
('i', 'cant')       -> 155 volte
('so', 'much')      -> 153 volte
LETTERARIO:
('to', 'be')        -> 602 volte
('of', 'the')       -> 563 volte
('in', 'the')       -> 442 volte
('it', 'was')       -> 418 volte
('i', 'am')         -> 363 volte
('she', 'had')      -> 322 volte
('she', 'was')      -> 317 volte
('had', 'been')     -> 305 volte
('it', 'is')        -> 281 volte
('could', 'not')    -> 278 volte

=== TRIGRAMMI (top 5) ===
TWITTER:
('i', 'love', 'you')    -> 84 volte
('you', 'so', 'much')   -> 74 volte
('thanks', 'for', 'the') -> 67 volte
('i', 'want', 'to')     -> 63 volte
('follow', 'amp', 'follow') -> 62 volte
LETTERARIO:
('i', 'do', 'not')      -> 123 volte
('i', 'am', 'sure')     -> 97 volte
('she', 'could', 'not') -> 68 volte
('a', 'great', 'deal')  -> 63 volte
('i', 'can', 'not')     -> 61 volte
```

Con le stopwords include dominano i costrutti grammaticali comuni ("to be", "of the") invece dei temi specifici — motivo per cui la sezione 4 usava il filtro.

6. Addestramento dei modelli n-gram: MLE e Laplace (righe 122–143)

Quattro modelli addestrati in totale: MLE e Laplace, ciascuno sia sul dominio Twitter sia su quello letterario.

Riga	Codice	Spiegazione
123	<pre>n = 2 # bigrammi</pre>	Ogni parola predetta condizionandola su una sola parola di contesto precedente.
126	<pre>train_tw_mle, vocab_tw_mle = padded_everygram_pipeline(n, sequenze_tweet)</pre>	Aggiunge marcatori <s>/</s> e genera gli everygram di addestramento dalle sequenze separate.
133	<pre># i generatori Python si esauriscono dopo il primo uso</pre>	Per addestrare Laplace serve richiamare la pipeline da capo: i generatori sono "usa e getta".
137	<pre>modello_tweet_lp = Laplace(n) # +1 fittizio a ogni conteggio</pre>	$P(w c) = (\text{count}(c,w)+1) / (\text{count}(c)+ V)$: riserva probabilità anche a combinazioni mai osservate.

► Esempio di output reale

```
Vocabolario Twitter: 12027 parole (12024 uniche + <s>/</s>/<UNK>)
Vocabolario Letterario: 9285 parole
```

7. Probabilità condizionali con MLE (righe 145–156)

Si interroga `.score(parola, [contesto])` per verificare, con esempi concreti, che ciascun modello abbia "imparato" le regolarità del proprio dominio.

Riga	Codice	Spiegazione
148–151	<pre>modello_tweet.score('love', ['i']) modello_tweet.score('mr', ['said'])</pre>	Su Twitter, coppie social attese (love i) vs una coppia letteraria (mr said) attesa nulla.
152–156	<pre>modello_lett.score('mr', ['said']) modello_lett.score('love', ['i'])</pre>	Speculare: sul modello letterario "mr said" attesa frequente, "i love" attesa rara.

► Esempio di output reale

Modello TWITTER:

```
P('love' | 'i') = 0.0530
P('wait' | 'cant') = 0.1146
P('happy' | 'so') = 0.0138
P('mr' | 'said') = 0.0000 <- bigramma mai osservato in questo corpus
```

Modello LETTERARIO:

```
P('mr' | 'said') = 0.0779
P('been' | 'had') = 0.1885
P('love' | 'i') = 0.0017
P('wait' | 'cant') = 0.0000 <- idem, mai osservato nel romanzo
```

Le due probabilità a 0.0000 sono il caso concreto che motiva lo smoothing di Laplace introdotto nella sezione 6: con MLE puro un bigramma mai visto vale esattamente zero.

8. Generazione di testo con MLE (righe 158–191)

I modelli n-gram possono anche **generare** testo campionando parola dopo parola secondo le probabilità apprese.

Riga	Codice	Spiegazione
162	<pre>testo_raw = modello.generate(n_parole, random_seed=42)</pre>	Campiona n_parole token in sequenza; il seed fisso rende la generazione riproducibile.
164–170	<pre>if token in ['<s>', '</s>', '<UNK>']: continue if len(token)==1 and token not in ['i', 'a']: continue</pre>	Scarta i marcatori di padding e i caratteri isolati (artefatti tipici di un bigramma povero di contesto).

► Esempio di output reale

Stile TWITTER (seed=42, 40 parole):

```
my app im glad you maybe great and makes doesnt even order it being sour kind
than players section wont be able to the original ticket sale at yes i love
you like to meet
```

Stile LETTERARIO (seed=42, 40 parole):

```
of a good deal of seeing women and is a few minutes be preferred by expressing
his seat the rest of coughing and his being acting to me therefore set off
with her long talking of the other
```

Confronto generazione per dominio (15 parole per seed):

```
seed=99 TWITTER: get calls and he will be gone think line up i hope you be there
seed=99 LETTERARIO: heard enough by harriet that he has not know who have found it can see
seed=123 TWITTER: i cant wait it for toffee do it can have
seed=123 LETTERARIO: but in all those attentions meaning change restlessness about him at first his
```

Grammaticalmente incoerente (i bigrammi non hanno memoria a lungo raggio) ma con un "sapore" lessicale riconoscibile del dominio di origine.

9. Perplexity cross-domain (righe 193–239)

La perplexity misura quanto un modello è "sorpreso" da una frase: bassa perplexity = frase prevedibile. Si usa come classificatore: si assegna la frase al modello con perplexity più bassa.

Riga	Codice	Spiegazione
198	<code>ngr = list(ngrams(pad_both_ends(tokens, n=n), n))</code>	<code>perplexity()</code> vuole tuple di n-grammi, non token grezzi: senza conversione NLTK indicizzerebbe i caratteri delle stringhe.
227–228	<code>pp_tw_n = pp_tw / len(modello_tweet_lp.vocab)</code>	Normalizzazione empirica per dimensione del vocabolario, per rendere confrontabili domini di taglia molto diversa.
231	<code>verdetto = "TW" if pp_tw_n < pp_lett_n else "LT"</code>	Vince il modello con perplexity normalizzata più bassa.

► Esempio di output reale

Frase	PP Twitter	PP Lett.	Verdetto
tw_pos_1	128.1	538.7	TW (corretto)
tw_pos_2	1425.3	3434.1	TW (corretto)
tw_neg_1	235.7	2151.0	TW (corretto)
tw_neg_2	577.8	1375.3	TW (corretto)
lett_1	6321.1	290.1	LT (corretto)
lett_2	797.7	159.3	LT (corretto)
lett_3	3821.2	442.0	LT (corretto)
lett_4	2605.8	126.2	LT (corretto)

Accuratezza classificatore: 8/8 = 100%

10. Classificatore basato su log-likelihood (righe 241–289)

Classificatore alternativo, ritenuto più stabile della perplexity: somma dei log delle probabilità dei bigrammi, divisa per il numero di transizioni.

Riga	Codice	Spiegazione
255–256	<code>if prob > 0: score += math.log(prob)</code>	Somma i log invece di moltiplicare le probabilità, evitando l'underflow numerico.
260	<code>return score / (len(tokens) - 1)</code>	Media per token: rende il punteggio comparabile fra frasi di lunghezza diversa.
268	<code>verdetto = "TWITTER" if ll_tw > ll_lett else "LETTERARIO"</code>	Vince il modello con log-likelihood più alta (meno negativa).

► Esempio di output reale

```
Testo: 'i love you so much thank you'
Log-lik Twitter: -5.289  Log-lik Letterario: -7.138  -> TWITTER (diff 1.85)

Testo: 'she had been very well disposed towards him'
Log-lik Twitter: -9.205  Log-lik Letterario: -6.143  -> LETTERARIO (diff 3.06)

Testo: 'mr knightley had always been very fond of her'
Log-lik Twitter: -9.062  Log-lik Letterario: -5.882  -> LETTERARIO (diff 3.18)

Testo: 'i cant stop smiling so happy'
Log-lik Twitter: -7.662  Log-lik Letterario: -8.890  -> TWITTER (diff 1.23)
```

Tutte le 8 frasi di test (4 mostrate qui) vengono classificate nel dominio atteso, con margini di confidenza variabili (da 1.23 a 3.18).

11. MLE vs Laplace su una parola mai vista (righe 291–305)

Esperimento mirato a mostrare esplicitamente il problema pratico di MLE con dati out-of-vocabulary, e come lo smoothing di Laplace lo risolva.

Riga	Codice	Spiegazione
294	<code>parola_rara = "xyzword"</code>	Stringa scelta apposta perché non compare in nessuno dei due corpora.
299	<code># MLE: parola ignota -> probabilità zero - > perplexity infinita</code>	Se un solo n-gramma ha probabilità 0, la perplexity dell'intera frase diverge a infinito.

► Esempio di output reale

```
Parola mai vista: 'xyzword'  
MLE      P = 0.000000  
Laplace P = 0.000065
```

```
Perplexity MLE      su frase con parola rara: inf  
Perplexity Laplace su frase con parola rara: 1111.48
```

Con MLE la perplexity diventa "inf" (inutilizzabile per un confronto numerico); Laplace resta un numero finito grazie al +1 di smoothing.

12. Confronto tra ordini n=1, n=2, n=3 (righe 307–323)

Si addestrano modelli Laplace di ordine crescente sul solo dominio Twitter, misurando la perplexity su una frase Twitter e una letteraria.

Riga	Codice	Spiegazione
312	<code># n crescente = modello più specifico</code>	La PP letteraria (fuori dominio) cresce molto più in fretta di quella Twitter (in dominio) al crescere di n.
315	<code>mod_tmp = Laplace(n_test)</code>	Laplace necessario: a n crescente aumenta la probabilità di trigrammi mai visti nella frase letteraria.

► Esempio di output reale

```
n=1: PP Twitter=124.04  PP Letterario=1727.63  
n=2: PP Twitter=128.14  PP Letterario=8059.42  
n=3: PP Twitter=307.20  PP Letterario=10003.72
```

La PP sulla frase letteraria cresce di oltre 5x passando da n=1 a n=3, mentre quella Twitter solo di ~2.5x: il modello diventa sempre più specifico al proprio dominio di training.

13. Analisi linguistica: TTR e lunghezza media (righe 325–350)

Metriche descrittive indipendenti dai modelli n-gram, per caratterizzare quantitativamente varietà lessicale e lunghezza sintattica.

Riga	Codice	Spiegazione
328	<code>N = min(len(tokens_tweet), len(tokens_lett))</code>	Tronca al campione più piccolo: il TTR è sensibile alla lunghezza del testo, va confrontato a parità di dimensione.

► Esempio di output reale

```
Type-Token Ratio (su campioni di 94706 token):  
Twitter:    0.1270  
Letterario: 0.0719  
-> Twitter ha vocabolario più vario (slang, abbreviazioni)
```

```
Lunghezza media:  
Tweet:      16.0 parole  
Frase letteraria: 25.6 parole  
-> Frasi letterarie 9.6 parole più lunghe in media
```

14. Generazione dei grafici riassuntivi (righe 352–458)

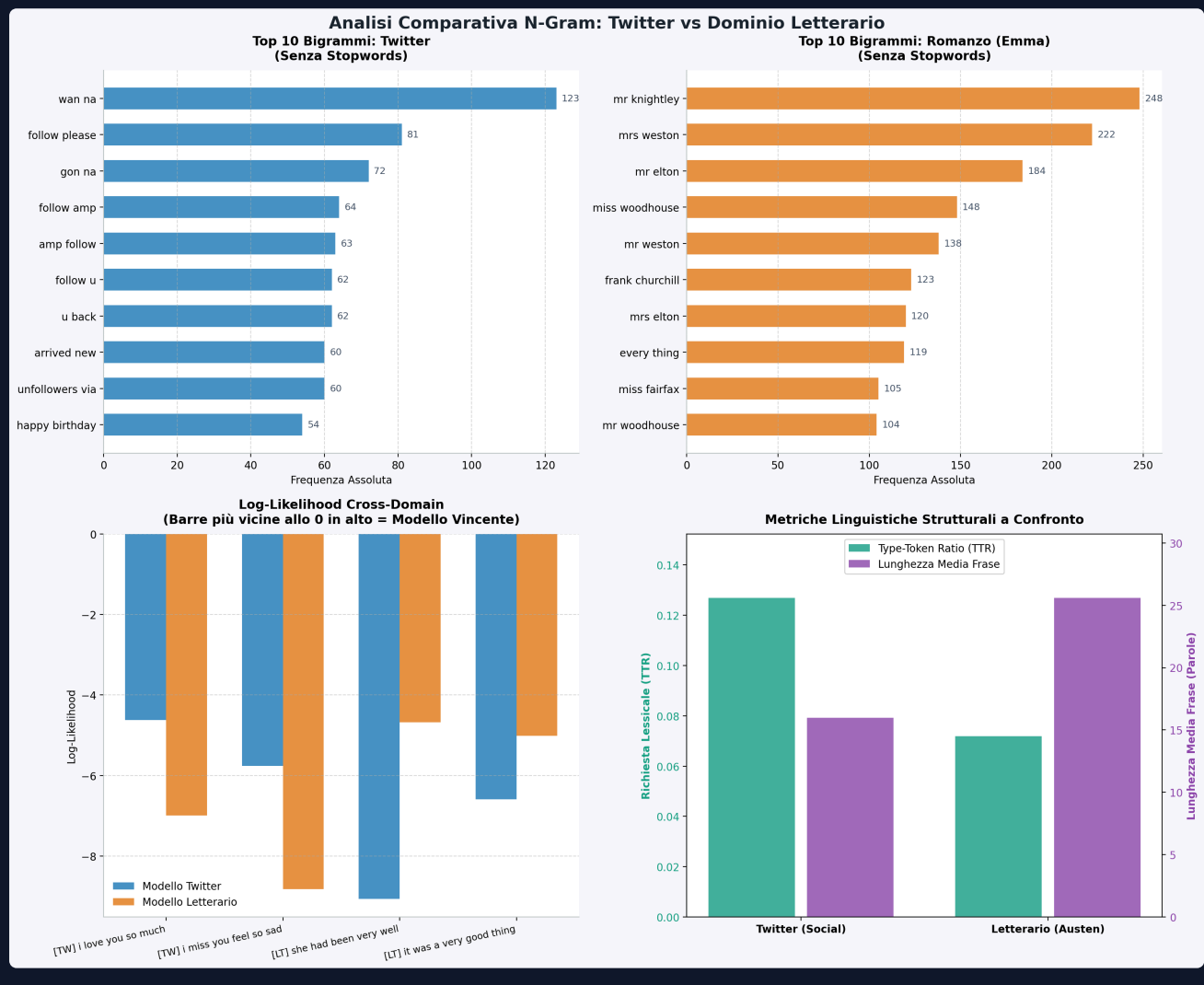
Tutti i risultati numerici vengono riassunti in un'unica figura con 4 pannelli (griglia 2×2), salvata come PNG, protetta da un blocco try/except.

Riga	Codice	Spiegazione
353	<code>try:</code>	Isola la generazione grafica: un errore qui non deve invalidare l'analisi già completata.
450–451	<code>cartella_script = os.path.dirname(os.path.abspath(__file__))</code>	Salva sempre accanto allo script, indipendentemente dalla directory di lancio.

► Esempio di output reale

Grafico salvato con successo in:

C:\Users\alless\Desktop\Unito-Magistrale-Informatica\TLN\Parte2Radicioni\confronto_risultati_laboratorio.png



15. Riepilogo finale (righe 460–478)

Riepilogo testuale compatto di tutti i numeri e le conclusioni chiave, leggibile senza scorrere tutto l'output precedente.

Riga	Codice	Spiegazione
473–476	<pre>print(" MLE -> generazione testo e probabilità condizionali") print(" Laplace -> perplexity (gestisce parole mai viste)") print(" Log-lik -> classificatore (più stabile di perplexity)")</pre>	Sintetizza il "ruolo" attribuito a ciascuna tecnica lungo lo script.

► Esempio di output reale

Dataset:

```
Twitter:    10000 tweet (5000 pos + 5000 neg)
Letterario: Emma by Jane Austen (1816)
```

Vocabolario:

```
Twitter:    12024 parole uniche
Letterario: 9305 parole uniche
```

TTR: Twitter=0.1270, Letterario=0.0719

Lunghezza media: Tweet=16.0, Letterario=25.6

Modelli:

```
MLE      -> generazione testo e probabilità condizionali
Laplace  -> perplexity (gestisce parole mai viste)
Log-lik  -> classificatore (più stabile di perplexity)
```

Lab completato!

Glossario dei concetti chiave

Riepilogo dei concetti NLP/statistici usati nel codice.

N-gramma

Sequenza di N token consecutivi. Bigramma (N=2): coppie adiacenti; trigramma (N=3): triple.

MLE (Maximum Likelihood Estimation)

$P(w|c) = \text{count}(c,w) / \text{count}(c)$. Non generalizza: eventi mai osservati hanno probabilità zero.

Smoothing di Laplace (add-one)

$P(w|c) = (\text{count}(c,w) + 1) / (\text{count}(c) + |V|)$. Garantisce che nessun n-gramma abbia probabilità esattamente zero.

Padding (<s>, </s>)

Marcatori artificiali di inizio/fine sequenza, permettono di stimare anche $P(\text{prima parola})$ e $P(\text{ultima parola})$.

Perplexity

Inverso della media geometrica delle probabilità assegnate agli n-grammi. Bassa = testo prevedibile; alta/infinita = testo sorprendente/fuori distribuzione.

Log-likelihood

Media dei logaritmi delle probabilità condizionali; evita l'underflow numerico rispetto a moltiplicare direttamente le probabilità.

Type-Token Ratio (TTR)

Parole uniche / parole totali; misura la varietà lessicale, va calcolato su campioni di dimensione uguale.

Catena di Markov di ordine 1

$P(\text{parola}_i | \text{storia}) \approx P(\text{parola}_i | \text{parola}_{i-1})$: l'assunzione alla base di un modello bigramma.